Static Website Hosting on AWS using Terraform

Complete Beginner-Friendly Explanation Guide

This PDF explains every important Terraform resource used in the project, why it is required, what happens if it is not used, and how AWS services work together. The project hosts a static website using AWS S3 and CloudFront CDN.

1. Terraform Block

Purpose: Defines required providers and versions.

Why used?
Terraform needs providers to communicate with AWS services.

Without it:
Terraform will not know which cloud provider to use.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.45.0"
    }
  }
}
```

---

2. AWS Provider Configuration

Purpose: Connects Terraform with AWS account and selected region.

Why used?
All resources will be created inside this AWS region.

Without it:
Terraform cannot deploy resources in AWS.

```
provider "aws" {
  region = var.aws_region
}
```

---

3. Random ID Resource

Purpose: Generates random characters for unique S3 bucket names.

Why used?
S3 bucket names must be globally unique across AWS.

Without it:
Bucket creation may fail because the name already exists.

```
resource "random_id" "bucket_id" {
  byte_length = 4
}
```

---

4. S3 Bucket

Purpose: Stores HTML, CSS, and JavaScript website files.

Why used?
S3 acts as static file storage for websites.

Without it:
No location exists to store website files.

```
resource "aws_s3_bucket" "static_website_bucket" {
  bucket = "portfolio-website-${random_id.bucket_id.hex}"
}
```

---

5. Website Configuration

Purpose: Enables static website hosting on S3 bucket.

Why used?
Allows S3 bucket to behave like a website server.

Without it:
S3 bucket only works as storage, not as a website.

```
resource "aws_s3_bucket_website_configuration" "website_config" {
  bucket = aws_s3_bucket.static_website_bucket.id

  index_document {
    suffix = "index.html"
  }
}
```

---

6. Public Access Block

Purpose: Controls whether bucket can be public or private.

Why used?
AWS blocks public access by default. Static websites need public read access.

Without it:
Users may receive 403 Access Denied errors.

```
resource "aws_s3_bucket_public_access_block" "public_access_block" {
  bucket = aws_s3_bucket.static_website_bucket.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}
```

---

7. Bucket Policy

Purpose: Allows internet users to read website files.

Important Concepts:
Principal = "*" means everyone.
s3:GetObject means read/download access.

Without it:
Website files cannot be opened publicly.

```
resource "aws_s3_bucket_policy" "bucket_policy" {
  bucket = aws_s3_bucket.static_website_bucket.id
}
```

---

8. Uploading Website Files

Purpose: Uploads all website files automatically to S3 bucket.

Important Features:
• for_each uploads multiple files automatically
• fileset() reads files from folder
• etag detects file changes
• content_type sets correct MIME types

Without it:
Files must be uploaded manually.

```
resource "aws_s3_object" "website_files" {
  for_each = fileset("${path.module}/www", "**/*")
}
```

---

9. CloudFront Distribution

Purpose: Creates CDN for faster global website delivery.

Benefits:
• HTTPS support
• Faster website loading
• Global caching
• Reduced latency

Without it:
Website may load slowly for distant users.

```
resource "aws_cloudfront_distribution" "cdn" {
  enabled = true
}
```

---

10. Cache Behavior

Purpose: Defines how CloudFront handles requests.

GET: Downloads files.
HEAD: Fetches metadata only.

viewer_protocol_policy:
Automatically redirects HTTP traffic to HTTPS.

```
default_cache_behavior {
  allowed_methods = ["GET", "HEAD"]
  cached_methods  = ["GET", "HEAD"]
}
```

---

11. Outputs

Purpose: Displays important deployment values after Terraform apply.

Why used?
Quickly access generated CloudFront website URL.

```
output "website_url" {
  value = "https://${aws_cloudfront_distribution.cdn.domain_name}"
}
```

---

12. Terraform Commands Workflow

- terraform init → Downloads providers and plugins
- terraform validate → Checks Terraform syntax
- terraform plan → Shows resources to be created

- terraform apply → Deploys infrastructure
- terraform destroy → Removes all created resources

Final Summary

This project demonstrates modern static website hosting architecture using: AWS S3 for file storage CloudFront CDN for fast global delivery Terraform for Infrastructure as Code (IaC) This project is highly valuable for DevOps beginners because it teaches AWS fundamentals, Terraform basics, CDN concepts, public access management, and infrastructure automation.